カリー=ハワード同型対応

―― 証明はプログラムであり,命題は型である ――

topazus 解説書 別冊

目次

1 一言でいうと 1
2 準備:型を「値の集まり」と見る 2
3 ならば($P \to Q$)は関数 2
4 かつ($P \land Q$)はタプル 3
5 または($P \lor Q$)は enum 3
6 真($\top$)と偽($\bot$) 4
7 すべての($\forall$)は依存関数 5
8 存在する($\exists$)は依存ペア 5
9 等しい($=$)も型 6
10 帰納法は再帰 6
11 推論規則との対応 7
12 排中律と構成的論理 8
13 歴史 8
14 topazus の設計への影響 9
15 練習問題 9
16 参考資料 10

1 一言でいうと

カリー=ハワード同型対応(Curry-Howard correspondence)は,**論理学**と**プログラミング**が,実は同じものを別の言葉で語っていた,という発見である.

論理学		プログラミング
命題	↔	型
証明	↔	その型を持つ値(プログラム)
証明が正しいか確かめる	↔	型検査

この対応のおかげで,「定理を証明する」ことを「プログラムを書く」ことに置き換えられる.そして証明の正しさは,コンパイラの型検査が確かめてくれる.Lean,Rocq(Coq),Agda,そして topazus が「証明できる言語」である理由はここにある.

この文書では,論理の記号を 1 つずつ取り上げて,それがプログラムの何に当たるかを見ていく.コード例は,実際に証明が書ける言語として **Lean 4** を使う.topazus での書き方は,まだ決まっていない部分が多いので,案として示す.

2 準備:型を「値の集まり」と見る

型は「その型を持つ値の集まり」と考えることができる.

- `bool` には `True` と `False` の 2 つの値がある.
- `i64` には -2^{63} から $2^{63}-1$ までの値がある.
- 値が 1 つしかない型も作れる.
- 値が **1 つもない**型も作れる.

ここで,次のように読み替えてみる.

> **読み替え**
> 型 `P` に値が 1 つでもある $\leftrightarrow$ 命題 P が証明できる(正しい)
> 型 `P` に値が 1 つもない $\leftrightarrow$ 命題 P は証明できない

その型の値を実際に 1 つ作ってみせることが,命題の証明になる.値の中身が何であるかは問題ではなく,**存在すること**が大事である.この見方で,論理の記号を 1 つずつ見ていく.

3 ならば($P \to Q$)は関数

3.1 対応

「P ならば Q」の証明とは何か.それは「P の証明を渡されたら,Q の証明を作ってみせる方法」である.これはまさに**関数**である.

$$\text{命題 } P \to Q \leftrightarrow \text{型 } \texttt{P -> Q}(\texttt{P} \text{ を受け取って } \texttt{Q} \text{ を返す関数})$$

3.2 例 1:$P \to P$

「P ならば P」は当たり前に正しい.証明は「受け取ったものをそのまま返す関数」,つまり恒等関数である.

```
-- Lean 4
theorem self_imp (P : Prop) : P → P :=
  fun hp => hp
```

`fun hp => hp` は「`hp` を受け取って `hp` を返す」無名関数である.`hp` は「P の証明」(hypothesis of P)という気持ちで名前を付けている.

3.3 例 2:三段論法は関数合成

「P ならば Q」と「Q ならば R」から「P ならば R」が言える.これは三段論法と呼ばれる,論理の基本である.

```
theorem syllogism (P Q R : Prop) : (P → Q) → (Q → R) → (P → R) :=
  fun f g => fun p => g (f p)
```

証明は「`p` に `f` を適用し,その結果に `g` を適用する」関数になっている.これは**関数合成**そのものである.topazus の `synth` で書けば次のようになる.

```
(synth [fn h] <- g, f)   // h = g ∘ f
```

> **ポイント**
> 三段論法と関数合成は同じものである.論理学者が 2000 年以上使ってきた推論の形と,プログラマが毎日使っている関数合成が,同じ構造を持っている.これがカリー=ハワード同型対応の面白さである.

3.4 仮定を使う

関数の引数は,証明における「仮定」に当たる.「P と仮定すると…」と言って議論を始めることは,`fun hp => ...` と書いて引数 `hp` を受け取ることと同じである.

4 かつ($P \wedge Q$)はタプル

4.1 対応

「P かつ Q」の証明とは,「P の証明」と「Q の証明」の**両方を持っていること**である.2 つの値を両方持つデータは,**タプル**(または struct)である.

$$\text{命題 } P \wedge Q \leftrightarrow \text{型 } \texttt{(P, Q)}\text{(ペア)}$$

4.2 例:$P \wedge Q \to Q \wedge P$

ペアを受け取って,中身を入れ替えたペアを返せばよい.

```
theorem and_swap (P Q : Prop) : P ∧ Q → Q ∧ P :=
  fun ⟨hp, hq⟩ => ⟨hq, hp⟩
```

`⟨hp, hq⟩` は Lean でペアを表す書き方である.topazus のタプルで書くと,`(hp, hq)` を受け取って `(hq, hp)` を返す関数になる.

5 または($P \vee Q$)は enum

5.1 対応

「P または Q」の証明とは,「P の証明」か「Q の証明」の**どちらか一方**と,それがどちらなのかの印である.これは **enum**(列挙型,直和型)である.

$$\text{命題 } P \vee Q \leftrightarrow \text{「}\texttt{P}\text{ の値」か「}\texttt{Q}\text{ の値」のどちらかを持つ enum}$$

Lean では `Or.inl hp`(P の証明)と `Or.inr hq`(Q の証明)の 2 種類がある.

5.2 場合分けは match

「P または Q」を仮定して何かを示すとき,数学では「P の場合」と「Q の場合」に**場合分け**をする.プログラムでは,これは **match** である.

```
theorem or_swap (P Q : Prop) : P ∨ Q → Q ∨ P :=
  fun h =>
    match h with
    | Or.inl hp => Or.inr hp
    | Or.inr hq => Or.inl hq
```

match の分岐が 1 つでも欠けると,型検査が通らない.これは「場合分けで 1 つの場合を検討し忘れた」証明が通らないことに対応する.

> **topazus への影響**
> topazus の enum はまだ「未定」になっている.カリー=ハワード同型対応の観点からは,enumと,すべての場合を書いたかを確かめる match(網羅性検査)は,「または」を扱うために必須の機能である.

6 真(⊤)と偽(⊥)

6.1 真は値が 1 つの型

「真」はいつでも証明できる命題である.証明として何の情報も要らないので,**値がちょうど 1 つの型**に対応する.この型は **unit 型**と呼ばれ,Rust の `()` に当たる.

6.2 偽は値が無い型

「偽」は決して証明できない命題である.これは**値が 1 つもない型(空の型)**に対応する.値が無いので,この型の値を作ることはできない.Rust の `!`(never 型)に当たる.

6.3 矛盾からは何でも言える

論理学には「矛盾からは何でも導ける」という規則がある(**爆発律**).プログラムで言うと,「空の型の値を受け取ったら,どんな型の値でも返せる」ということである.

```
theorem explode (P : Prop) : False → P :=
  fun h => nomatch h
```

空の型の値 `h` に match をかけると,分岐が 1 つも無い.書くべき分岐が無いので,どんな型の値を返すことにしても型検査が通る.そもそも空の型の値は存在しないので,この関数が実際に呼ばれることは無い.

6.4 否定は「偽への関数」

「P でない」($\neg P$)は,「P を仮定すると矛盾する」と定義される.

$$\neg P := P \to \bot$$

つまり否定の証明は,「P の証明を受け取ったら空の型の値を返す関数」である.P の証明が存在しない限り,こういう関数は矛盾なく書ける.

> **topazus の `[void]` について**
> topazus の `[void]` は「値を持たない」と説明されていて,返り値の無い関数に使われている.しかしカリー=ハワード同型対応の言葉では,「返り値の無い関数」の返り値は **unit 型**(値が 1 つ)であり,「値を持たない型」は**空の型**(値が 0 個)で,別物である.

C 言語の `void` は unit 型に近く,Haskell の `Void` は空の型である.名前が紛らわしいので,topazus では次のように分けるのがおすすめである.

- `[unit]`:値が 1 つ.返り値の無い関数に使う.命題としては「真」.
- `[never]`:値が 0 個.命題としては「偽」.

7 すべての(∀)は依存関数

7.1 対応

「すべての自然数 n について $P(n)$」の証明とは何か.それは「どんな n を渡されても,$P(n)$ の証明を返す方法」である.これは関数だが,**返り値の型が引数の値によって変わる**ところが普通の関数と違う.$n = 3$ を渡せば $P(3)$ の証明が,$n = 5$ を渡せば $P(5)$ の証明が返ってくる.

返り値の型が引数の値に依存するので,これを**依存関数型**(Π 型,パイ型)という.依存型の言語でなければ書けない型である.

$$\text{命題 } \forall n.P(n) \leftrightarrow \text{依存関数型 } \texttt{(n : Nat) → P n}$$

topazus の定理の `[∀ nat a, nat b]` の部分は,この依存関数の引数を表している.

```
(theorem add-comm [∀ nat a, nat b] (== (+ a b) (+ b a)) | ... |)
// 「a と b を受け取って,(+ a b) と (+ b a) が等しいことの証明を返す関数」
```

8 存在する(∃)は依存ペア

8.1 対応

「ある n が存在して $P(n)$」の証明とは,「実際の n の値」と「その n について $P(n)$ が成り立つ証明」のペアである.具体例を 1 つ示せば,存在の証明になる.

ペアの 2 つ目の型が 1 つ目の値に依存するので,これを**依存ペア型**(Σ 型,シグマ型)という.

```
theorem exists_gt_three : ∃ n : Nat, n > 3 :=
  ⟨4, by decide⟩
```

`⟨4, ...⟩` は「$n = 4$ がその例で,$4 > 3$ の証明は `by decide`(計算して確かめる)」という意味である.

topazus の篩型との関係

topazus の篩型 `[i64 n | (> n 0)]` は,「値 `n`」と「`n > 0` の証明」のペアとみなせる.つまり篩型は依存ペアの一種である.ただし証明の部分は数量 0 として実行時には消えるので,実行時には `n` だけが残る.

9 等しい(=)も型

9.1 等号の型

「$a = b$」も 1 つの型である.この型の値を作る方法はただ 1 つ,「同じものは等しい」(**反射律**,reflexivity)だけである.Lean では `rfl` と書く.

9.2 計算で証明する

型検査器は,必要なら式を計算してから比べる.そのため,$2 + 3 = 5$ のような命題は,計算すれば両辺が同じになるので `rfl` だけで証明できる.

```
example : 2 + 3 = 5 := rfl
```

「計算すれば分かること」は,型検査器が自動で確かめてくれる.依存型の型検査器が中に小さなインタプリタを持つ(解説書第 9 章)のはこのためである.

10 帰納法は再帰

10.1 自然数を定義する

ここからは,自然数を自分で定義して,性質を証明してみる.自然数は次の 2 つから作られる.

- `zero`:0
- `succ n`:n の次の数$(n + 1)$

たとえば 3 は `succ (succ (succ zero))` である.これを**ペアノの自然数**という.

```
inductive N where
  | zero : N
  | succ : N → N
```

足し算は,1 つ目の引数についての再帰で定義する.

```
def add : N → N → N
  | N.zero,   m => m                    -- 0 + m = m
  | N.succ n, m => N.succ (add n m)     -- (n+1) + m = (n + m) + 1
```

10.2 計算だけで分かる性質

$0 + m = m$ は,`add` の 1 行目がそのまま言っていることなので,計算だけで証明できる.

```
theorem zero_add (m : N) : add N.zero m = m := rfl
```

10.3 帰納法が必要な性質

ところが $n + 0 = n$ は計算だけでは証明できない.`add` は 1 つ目の引数で場合分けするので,1 つ目が変数 `n` のままでは計算が進まないからである.そこで n についての帰納法を使う.

- $n = 0$ のとき:$0 + 0 = 0$ は計算で分かる.
- $n = k + 1$ のとき:$k + 0 = k$ を仮定する(帰納法の仮定).すると $(k + 1) + 0 = (k + 0) + 1 = k + 1$.

これを Lean で書くと,**再帰関数**になる.

```
theorem add_zero : (n : N) → add n N.zero = n
  | N.zero   => rfl
  | N.succ k => congrArg N.succ (add_zero k)
```

2 行目で `add_zero k` と自分自身を呼んでいる.これが「帰納法の仮定を使う」ことに当たる.`congrArg N.succ` は「両辺に `succ` を付けても等しい」という意味である.

> **ポイント**
> 数学的帰納法は再帰関数である.「0 の場合」が再帰の終了条件で,「k の場合から $k+1$ の場合を示す」が再帰呼び出しに当たる.

10.4 なぜ停止性が必要か

再帰関数が証明になるのは,**その再帰が必ず終わる**からである.`add_zero (N.succ k)` は `add_zero k` を呼び,引数はいつか `N.zero` にたどり着く.

もし終わらない再帰を許すと,次のような「証明」が書けてしまう.

```
-- (Lean ではエラーになる)
theorem bad (P : Prop) : P := bad P
```

型だけ見れば「どんな命題 P でも証明できる」と言っているが,実際には自分を呼び続けるだけで何も作っていない.これを許すと $1 = 2$ も「証明」できてしまうので,証明に使う関数は停止性を検査する必要がある.topazus で `partial` な関数を証明に使えないのはこのためである.

11 推論規則との対応

11.1 導入と除去

論理学では,証明の規則を**推論規則**として書く.線の上が前提,下が結論である.各記号には「それを作る規則(導入)」と「それを使う規則(除去)」がある.

$$\frac{P \quad Q}{P \wedge Q}\ \wedge\text{導入} \qquad \frac{P \wedge Q}{P}\ \wedge\text{除去} \qquad \frac{P \to Q \quad P}{Q}\ \to\text{除去}$$

これをプログラムの言葉で読むと次のようになる.

記号	導入(作る)	除去(使う)
$P \to Q$	無名関数を作る	関数を呼ぶ
$P \wedge Q$	ペアを作る	ペアから取り出す
$P \vee Q$	enum の値を作る	match で場合分け
$\bot$	(作れない)	空の match
$\forall x.P(x)$	依存関数を作る	具体的な値を渡す
$\exists x.P(x)$	値と証明のペアを作る	ペアから取り出す
$a = b$	`rfl`	等しいものを置き換える

導入規則はデータの作り方,除去規則はデータの使い方に対応している.論理学の推論規則と,プログラミング言語の型付け規則は,1 対 1 に対応する.

12 排中律と構成的論理

12.1 証明できない「当たり前」

普通の数学では,「P か,P でないかのどちらかである」($P \vee \neg P$,**排中律**)は当たり前に正しいとされる.

ところがカリー=ハワード同型対応の世界では,これは一般には証明できない.$P \vee \neg P$ の証明とは,enum の値,つまり「P の証明」か「$\neg P$ の証明」の**どちらかを実際に作ってみせる**ことだからである.任意の命題 P について,どちらが正しいかを判定するプログラムは書けない(たとえば P が「このプログラムは停止する」なら,停止性問題になる).

12.2 構成的論理

このように「実際に作ってみせる」ことを証明とする論理を,**構成的論理**(直観主義論理)という.カリー=ハワード同型対応が素直に当てはまるのは,この論理である.

- 構成的な証明は,そのままプログラムとして**実行できる**.たとえば「解が存在する」という構成的証明からは,実際に解を計算するプログラムが取り出せる.
- 排中律を使う証明は,実行できない部分を含む.

Lean は,排中律を**公理**として追加することで,普通の数学も扱えるようにしている.公理は「証明なしで正しいとみなす値」なので,実行はできない.

> **topazus への影響**
> topazus では証明は実行時に消える(数量 0)ので,証明に排中律を使っても実行には影響しない.そのため,Lean のように排中律を公理として用意してもよい.ただし数量 0 の場所でしか使えないようにする必要がある.

13 歴史

年	出来事
1930 年代	ブラウワー,ハイティング,コルモゴロフが「証明とは構成である」という解釈を考える(BHK 解釈)
1934 年	カリーが,コンビネータ(関数の組み合わせ)の型と,論理の公理の形が同じであることに気づく
1969 年	ハワードが,型付きラムダ計算と自然演繹(推論規則による証明)の対応を示す原稿を書く(出版は 1980 年)
1970 年代	マーティン=レーフが,依存型を持つ型理論を提案する.$\forall$ と $\exists$ も型で表せるようになる
1980 年代	コカンとユエが構成計算(Calculus of Constructions)を提案.これをもとに Coq(現在の Rocq)が作られる
2000 年代〜	Agda,Idris,Lean などが登場.証明をプログラムとして書くことが実用的になる

「カリー=ハワード」という名前は,この対応を見つけた2人の名前から来ている.依存型まで含めて「カリー=ハワード=ランベック対応」と呼ぶこともある(ランベックは圏論との対応を示した).

14 topazus の設計への影響

カリー=ハワード同型対応から見ると,topazus で決めるべきことがはっきりする.

論理で必要なもの	topazus で必要な機能
ならば(→)	関数.すでにある
かつ(∧)	タプル・struct.タプルはある.struct は未定
または(∨)	enum と網羅的な match.未定
真・偽	`[unit]` と `[never]` の区別.今の `[void]` を見直す
すべての(∀)	依存関数.`theorem` の `[∀ ...]`
存在する(∃)	依存ペア.篩型がその一種
等しい(=)	等号の型と `rfl` に当たるもの
帰納法	停止性が検査された再帰関数

また,この対応から見ると**定理は特別なものではない**.`theorem` は「型が命題で,数量 0 の値を `let` で定義すること」と同じである.

```
(theorem add-comm [∀ nat a, nat b] (== (+ a b) (+ b a)) | proof |)

// 上は次と同じ意味(案)
(let 0 [fn add-comm] <- [(== (+ a b) (+ b a)) r] [nat a, nat b] -> | proof |)
```

`theorem` を `let` の書き換え(糖衣構文)として実装すれば,型検査器に「証明専用の仕組み」を作らなくて済む.関数の型検査がそのまま証明の検査になる.

15 練習問題

次の命題を,Lean 4 で証明してみよう.どれも関数・ペア・match だけで書ける.

1. $P \to Q \to P$
2. $P \land Q \to P$
3. $P \to P \lor Q$
4. $(P \to Q) \to \neg Q \to \neg P$(対偶)
5. $\neg\neg(P \lor \neg P)$(排中律の二重否定.排中律そのものは証明できないが,これは証明できる)

15.1 解答例

```
-- 1. Q の証明は使わずに捨てる
theorem ex1 (P Q : Prop) : P → Q → P :=
  fun hp _ => hp

-- 2. ペアの1つ目を取り出す
```

```
theorem ex2 (P Q : Prop) : P ∧ Q → P :=
  fun ⟨hp, _⟩ => hp

-- 3. enum の左側を作る
theorem ex3 (P Q : Prop) : P → P ∨ Q :=
  fun hp => Or.inl hp

-- 4. ¬Q は Q → False.P を受け取ったら f で Q にして nq に渡す
theorem ex4 (P Q : Prop) : (P → Q) → ¬Q → ¬P :=
  fun f nq => fun hp => nq (f hp)

-- 5. 「P ∨ ¬P が偽だ」という仮定 h から矛盾を作る
theorem ex5 (P : Prop) : ¬¬(P ∨ ¬P) :=
  fun h => h (Or.inr (fun hp => h (Or.inl hp)))
```

4 番は三段論法(関数合成)の応用である.5 番は少し難しいが,「仮定 h を 2 回使う」ところがポイントになっている.

16 参考資料

- Theorem Proving in Lean 4:Lean 公式の教材.「Propositions and Proofs」の章がこの文書の内容に当たる.
- Philip Wadler, Propositions as Types (2015):カリー=ハワード同型対応の歴史と考え方を一般向けに解説した論文.読みやすい.
- 『型システム入門』(Benjamin C. Pierce):型付きラムダ計算の教科書.
- Software Foundations 第 1 巻 Logical Foundations:Rocq で論理と証明を学ぶ教材.